BCSE301P SOFTWARE ENGINEERING LAB

CyberGuard AI

Intelligent Cyber Threat Detection System

Viva Preparation Guide

What this document is

A single-sitting preparation guide for the CyberGuard AI viva. It explains what the system does, how each part works, how the implementation maps onto every Software Engineering Lab experiment, a timed live-demonstration script, and prepared answers to the questions most likely to be asked. Every number quoted here was measured from the running system, not estimated.

Delivered	Figure	Delivered	Figure
Backend Python modules	48	**Automated tests**	117 (all passing)
Frontend modules	22	**REST endpoints**	46
Database entities	10	**Frontend screens**	12
Detection categories	8	**Engineered features**	18
Behavioural rules	7	**SE documents**	23

Generated 22 September 2026

Contents

#	Section	Why it matters
1	The project in sixty seconds	What to say when asked 'what is it?'
2	Architecture	Layers, dependency rule, why it is shaped this way
3	The detection engine	The technical core - expect the most questions here
4	Database design	Ten entities, the relationships that matter
5	The incident state machine	The strongest single demonstration
6	Syllabus mapping	Every experiment traced to an artefact
7	The application in twelve screens	Real captures from the running system
8	Live demonstration script	A timed ten-minute walkthrough
9	Anticipated viva questions	Thirty questions with prepared answers
10	Defects found and fixed	The honest engineering story
11	Limitations and future work	What to admit before being asked
12	Quick reference card	Commands, numbers and constants to memorise

How to use this guide

Read sections 1, 3 and 5 until you can deliver them without notes - they carry the most marks. Section 8 is the demonstration itself. Section 10 is what separates a project that was built from one that was merely assembled: be ready to talk about the defects you found, because being able to describe a bug you fixed is stronger evidence of understanding than any feature list.

1 The project in sixty seconds

CyberGuard AI is a working Security Operations Centre platform. It takes security telemetry - records of network connections and authentication attempts - classifies each record with an explainable detection engine, raises alerts for anything suspicious, drives an investigation through a guarded lifecycle, and produces a report at the end.

The distinguishing design decision is that **every verdict is explainable**. The engine never returns a bare score. It returns a classification, a risk value from 0 to 100, a confidence, the individual contribution of each of its three detectors, and a list of plain-English reasons an analyst can act on or challenge.

The second decision is that **the documentation and the code are the same system**. The state transition diagram is the transition table the API enforces. The ER diagram is the SQLAlchemy schema. The severity bands in the Level 2 data flow diagram are the constants the engine applies. None of the diagrams were drawn separately from the implementation, which is why they cannot drift out of step with it.

What it deliberately does not do

The system is strictly defensive and that is visible at the diagram level. Looking at the context diagram, **there is no outbound arrow to any network, host or third-party service**. CyberGuard AI only ever replies to the people who call it. It cannot scan, probe or attack anything, because it has no data flow through which to do so. All telemetry is pushed in by a human - either typed into a form or uploaded as a CSV file.

- No malware, exploits or payloads of any kind exist in the repository.
- No credential harvesting, password cracking or brute-force capability.
- No scanning or attack automation - the engine only reads numbers it was handed.
- No real threat infrastructure: indicators use the RFC 5737 documentation ranges (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24) and the reserved *.invalid* and *.example* domain namespaces.

> **Say this if asked 'is this safe?'**
> "It is defensive by construction, not by policy. The context diagram has no outbound flow - the system has no mechanism to reach any host. It reads numbers that a user gives it and scores them. Every address in the demonstration data comes from the IETF ranges reserved for documentation."

2 Architecture

```
Browser  -  React 18 + Vite + Tailwind + Recharts
    |
    |   REST / JSON, Bearer token
    v
Flask application factory
    |
    +-- routes/     11 blueprints, 46 endpoints, one JSON envelope
    +-- services/   detection, alert, incident, analytics, report,
    |               auth, audit
    +-- ml/         detection engine
    |                 features.py   18 engineered features
    |                 rules.py      7 explainable rules
    |                 anomaly.py    statistical baseline
    |                 detector.py   fusion, severity, explanation
    +-- models/     10 SQLAlchemy entities
    +-- utils/      validation, security, errors, CSV, logging
    |
    v
SQLite  -  database/cyberguard.db
```

The dependency rule

Dependencies point strictly inward and never back out: `routes` → `services` → `models` → `extensions`, with `utils` available to every layer.

Package	May import	Must never import
routes	services, utils, models	nothing imports routes
services	models, ml, utils, extensions	routes
ml	standard library, NumPy, scikit-learn (optional)	models, services, routes, Flask, the database
models	extensions, utils	services, routes, ml
utils	models (inside a function body only)	services, routes

> **The point worth making about ml/**
> The detection engine imports nothing from Flask, from SQLAlchemy or from the rest of the application. That is a deliberate design decision with three payoffs: its 18 unit tests run with no database and no HTTP layer; the same engine runs from the command line via `python -m app.ml.predict`; and it could be swapped for a different implementation provided `detect()` keeps its contract.

Why these technology choices

Choice	Reason
SQLite, not PostgreSQL	No service to install and no credentials to configure - the whole database is one file that can be deleted and rebuilt with a single command. The lab machine may not permit installing a database service. SQLAlchemy abstracts the dialect, so moving to PostgreSQL is a change to one URL.
Flask, not Django	The project needs a REST API and nothing else. Django's admin, templating and auth stack would all be unused weight.
Detection engine in-process	Avoids a second process to start and a network hop per event. At the prototype's volumes an in-process call takes 7 milliseconds.

Choice	Reason
ML is optional	If scikit-learn or the trained artefacts are absent the engine reports `hybrid-rules` mode and keeps working. This is proved by a test, not merely claimed.
Vite dev-server proxy	The browser only ever talks to one origin during development, so CORS is never exercised. CORS is still configured for the case where the frontend is served separately.

3 The detection engine

This is the technical core of the project and where the most detailed questions will be asked. Three independent detectors vote on every event.

Detector	Nature	What it contributes
Rule bank	Deterministic, fully explainable	7 behavioural rules; each returns a score and a list of reasons
Statistical baseline	Unsupervised	Robust z-scores against learned normals, mapped through a saturating curve
Machine learning	Optional, supervised + novelty	RandomForest probability and IsolationForest outlier score

The eight processing stages

#	Stage	What happens
1	Extract features	11 observed fields become 18 numeric features
2	Rule bank	All 7 rules evaluated; highest score wins the classification
3	Statistical	z-scores; only deviations above +2 sigma count
4	Machine learning	Skipped entirely if artefacts are missing
5	Fuse scores	Weighted blend with a rule-score floor
6	Map severity	Risk band, and the threat/benign verdict
7	Confidence	Grows with signal strength and detector agreement
8	Assemble explanation	Reasons, recommended action, MITRE tactic, timing

The fusion formula

```
with ML:     weighted = 0.58*rule + 0.18*anomaly + 0.24*ml
without ML:  weighted = 0.74*rule + 0.26*anomaly

risk = clamp( max(weighted, 0.85 * rule_score) * sensitivity , 0, 100 )
```

The **rule score acts as a floor**. The statistical and machine-learning detectors contribute only through the weighted term, so on their own they can raise an event to MEDIUM for review but never to CRITICAL. This is a deliberate decision: an unexplained statistical outlier is worth a human glance, but it is not worth waking someone up. Without the floor the alert queue fills with unactionable 'critical' noise.

`sensitivity` is the analyst's own setting from the Settings page, clamped to the range 0.5 to 1.5.

Severity bands and what they trigger

Band	Risk	Consequence	Constant
LOW	0 - 25	Benign. Stored and counted, nothing raised.	
MEDIUM	26 - 50	Threat recorded and an **alert raised**.	alert threshold
HIGH	51 - 75	Threat and alert; analyst prioritises it.	
CRITICAL	76 - 100	Alert plus an **incident opened automatically**.	incident threshold

The seven rules

Rule	Fires when	Magnitude term
Brute Force Attempt	failed logins ≥ 5	38 + 30·log10(failed/4)
Denial-of-Service Pattern	requests ≥ 400 or packets/s ≥ 400	20 + 30·log10(requests/300)
Data Transfer Anomaly	bytes ≥ 45 MB	18 + 30·log10(MB/40)
Port Scanning Pattern	requests ≥ 25 and bytes/packet ≤ 220	18 + 26·log10(requests/20)
Credential Abuse Pattern	auth event, failed ≥ 3, requests ≥ 30	22 + 18·log10(requests/25), capped at 45
Suspicious Login Activity	1 ≤ failed logins ≤ 4	15 + 7·failed
Network Anomaly	catch-all traffic-shape rule	duration and packet-size terms

Why the magnitude terms are logarithmic - the best answer in the project

With a linear term, a brute-force attempt with 20 failed logins and one with 2000 both saturate at 100, so the entire alert queue collapses into a single severity band and the dashboard conveys nothing. A logarithmic curve keeps the ordering intact across three orders of magnitude: 5 failures score 41, 20 score 59, 50 score 71, 150 score 85. That is exactly what an analyst needs in order to prioritise. This was not the first design - see section 10, defect D1.

Measured performance

Measured on the development machine (Windows 11, Python 3.10.0) over 2000 generated events per mode, first call discarded:

Mode	Mean	Median	p95	p99	Throughput
hybrid-ml	7.083 ms	6.596 ms	9.890 ms	13.672 ms	141 events/s
hybrid-rules	0.033 ms	0.027 ms	0.069 ms	0.121 ms	29 930 events/s

The requirement was under 50 ms per event, so the p99 in ML mode sits comfortably inside it. The first measurement did not - see defect D3 in section 10.

Honesty about the model accuracy

> **Do not claim 99.9% accuracy - explain it**
>
> The trained classifier reports around 99.9% on its hold-out split, and that number is **not** evidence of real-world performance. The synthetic generator produces behaviour profiles that are well separated by construction, so a high score reflects that separability, not detection skill. The system has never analysed a real network capture. The metadata file, the project report and this guide all state this. Saying so first is far stronger than being caught by the question.

The dashboard therefore never displays a bare accuracy figure. It selects a source in priority order and labels it:

- **Analyst-confirmed precision** once five or more alerts have been triaged: resolved ÷ (resolved + false positives).
- Otherwise the **model hold-out accuracy**, explicitly marked as measured on synthetic data.
- Otherwise the **mean engine confidence**, marked as a fallback.

Every response carries `source`, `label` and `note`, so a number is never presented without saying where it came from.

4 Database design

Ten entities. The schema is in Third Normal Form with two documented, deliberate exceptions.

Entity	Purpose	Key relationship
User	Analysts and administrators	owns Incidents, AuditLogs, Reports
Asset	Monitored hosts and devices	1 : N SecurityEvent
SecurityEvent	One observed record - the central fact table	1 : 1 DetectionResult
DetectionResult	The engine's verdict	unique FK to SecurityEvent
Threat	A classified malicious-looking finding	N : 1 SecurityEvent
Alert	Analyst-facing notification	N : 1 Threat, N : 1 Incident
Incident	Investigation case with a lifecycle	1 : N Alert
ThreatIndicator	Simulated intelligence records	no FK - matched by value
Report	Generated security report	N : 1 User
AuditLog	Append-only trail of state changes	N : 1 User

The three relationships worth explaining

SecurityEvent → DetectionResult is the only true 1:1. It is enforced by a UNIQUE constraint on the foreign key. Conceptually one event cannot have two contradictory verdicts.

Incident → Alert is what makes triage work. Several alerts from the same source attach to one incident, so the analyst investigates a campaign rather than each packet-level finding in isolation. When the incident closes, every alert attached to it closes with it.

ThreatIndicator has no foreign key by design. Indicators are correlated to events by value, not by referential integrity. Adding a foreign key would mean an indicator could only exist once matching evidence already existed - which inverts how an intelligence feed actually works, since the feed is meant to be populated before the matching traffic appears.

The deliberate denormalisation

Be ready for this - an examiner who reads the schema will spot it. Severity and score are duplicated across DetectionResult, Threat and Alert. That is a **temporal snapshot**, not an oversight: an alert records the severity as assessed at the moment it was raised. If an analyst later changes their detection sensitivity, previously stored detections must keep their original scores, otherwise historical charts would silently rewrite themselves. For a security tool, auditability outweighs strict normalisation.

The second exception is JSON-serialised columns - reasons, features, history and report content. These are variable-length, always read whole with their parent row, and never queried or joined on. Five child tables would add joins to every read for no query benefit.

Deletion behaviour

Relationship	On delete	Why
SecurityEvent → DetectionResult / Threat / Alert	cascade delete	A verdict about a deleted event is meaningless
Asset → SecurityEvent	**set asset_id to NULL**	**Retiring an asset must not destroy security history**
Incident → Alert	no cascade	Alerts survive and can be re-linked

5 The incident state machine

> **This is the single strongest demonstration in the project**
> It shows a behavioural model that is not merely drawn but **enforced at runtime**. The transition table in the design document is the same Python dictionary the API consults on every request. Demonstrate it live - section 8, step 4.



The transition table - the executable specification

From state	Permitted next states
NEW	OPEN, INVESTIGATING, FALSE_POSITIVE
OPEN	INVESTIGATING, CONTAINED, FALSE_POSITIVE
INVESTIGATING	CONTAINED, RESOLVED, FALSE_POSITIVE
CONTAINED	RESOLVED, INVESTIGATING
RESOLVED	INVESTIGATING (reopen)
FALSE_POSITIVE	INVESTIGATING (reopen)

Anything not in that table is refused with **HTTP 409 CONFLICT**, and the response names the current state and the permitted set.

Why specific transitions are forbidden

Attempted	Why it is refused
NEW → CONTAINED	Nothing can be contained before anyone has looked at it
NEW → RESOLVED	Closing a case nobody investigated would defeat the audit trail
CONTAINED → FALSE_POSITIVE	Containment was already applied, so the finding was acted on; retroactively calling it a false positive would misstate the history
RESOLVED → CONTAINED	Cannot regress from closed - reopen to INVESTIGATING first

How the user interface stays honest

The API returns `allowed_transitions` on every incident read, and the Incident Details page renders a button only for each entry in that array. The interface therefore **cannot offer an illegal move** - and the API would refuse it anyway if something else called it directly. Two independent guards, one source of truth.

Why alerts have a different, looser machine

An alert may legitimately be resolved straight from NEW: an experienced analyst can recognise a known-benign pattern at a glance. An **incident**, by contrast, exists precisely because something needs work, so skipping investigation would make its audit trail meaningless. The asymmetry is the reason the two machines are separate rather than shared.

The feedback loop

The two terminal alert states are not merely bookkeeping - they are the system's ground-truth signal. Once five or more alerts have been closed, `resolved ÷ (resolved + false positives)` becomes the Detection Accuracy figure on the dashboard, labelled 'analyst-confirmed precision'. Triage decisions made in section 8 of the demonstration change the number shown in step 2.

6 Syllabus mapping

Every lab experiment traced to the artefact that satisfies it and to the code that implements it. All documents are in `documentation/`.

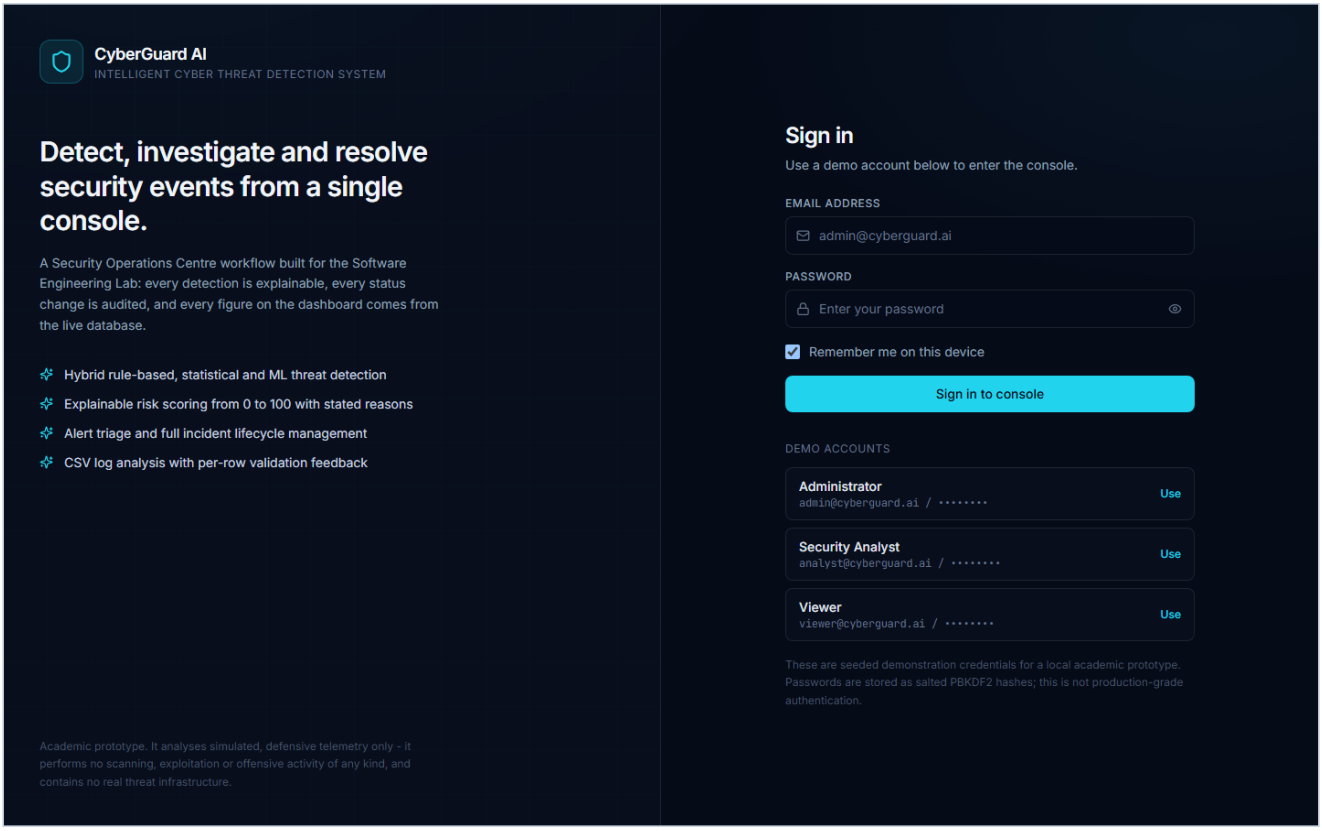
Experiment	Artefact	Doc	In the code
1. Process model	Incremental/Agile selected after comparing Waterfall, Prototyping, Spiral; 9 increments; risk register	01	Evidenced by the increment-7 re-calibration
2. WBS and estimation	All four views - process, product, geographic, role - with duration, effort, dependencies, roles	02	445-hour baseline, cross-checked
3. ER diagram	10 entities, 9 relationships, cardinality, keys, indexes, normalisation	04	app/models/ (10 files)
4. Context + DFD	Context, Level 0, Level 1, Level 2 with balancing checks	05-08	routes/, services/, ml/, csv_parser.py
5. State transition	Incident and alert lifecycles, legal and forbidden transitions	09	models/incident.py
6. Use case + class	3 actors, 17 use cases, include/extend; entity, control and service classes	10, 11	whole code base
7. Interaction models	7 sequence diagrams, 6 communication diagrams	12, 13	whole code base
8. Package / component / deployment	Dependency rule, provided-required interfaces, degradation matrix, two topologies	14-16	package layout of app/ and src/
9. Testing	117 automated cases, 11 manual scenarios, measured non-functional results	17-19	backend/tests/ (5 modules)
10. Storyboard + UI	9-frame narrative storyboard; colour system, components, accessibility	20, 21	frontend/src/

The concept-to-realisation table

Concept	CyberGuard AI realisation
Process Model	Incremental/Agile with a documentation-inclusive Definition of Done
WBS	Project task decomposition in four views
ERD	The database structure - 10 entities as SQLAlchemy models
DFD	Security event processing, down to the engine's eight stages
State Diagram	The incident lifecycle, enforced by the API
Use Case	Analyst and administrator interactions with the console
Class Diagram	Entity, service and engine classes with multiplicity
Sequence Diagram	The threat-detection interaction, end to end
Component Diagram	System architecture with its degradation paths
Deployment Diagram	The two-process local deployment on Windows 11
Testing	117 backend cases plus manual frontend scenarios
Storyboard	One analyst's shift, from login to handover report
UI Design	The Security Operations Centre dashboard

7 The application in twelve screens

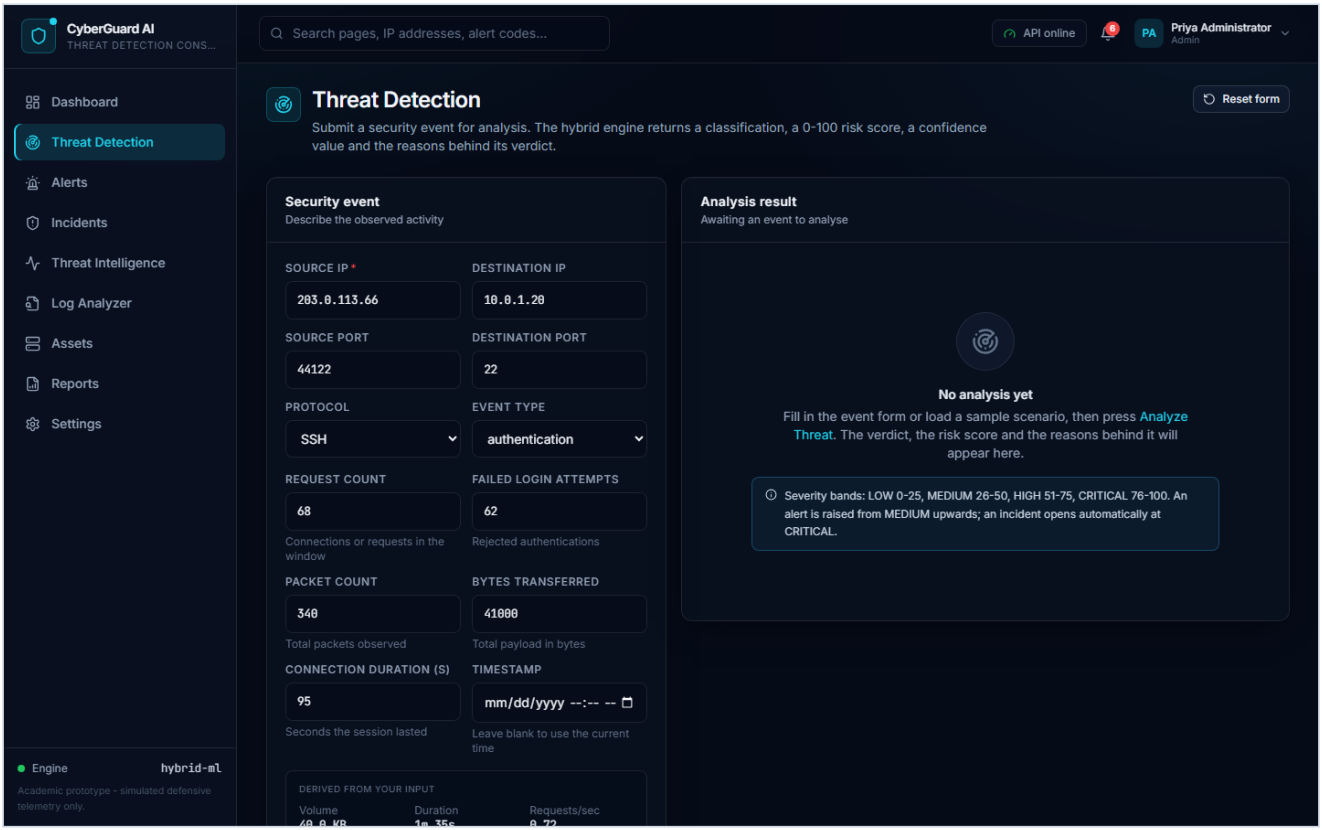
Every image below is a real capture from the running application at a 1440×900 viewport. Demonstration passwords on the login screen were replaced with bullet characters before capture; no credential, token or key appears in any image.



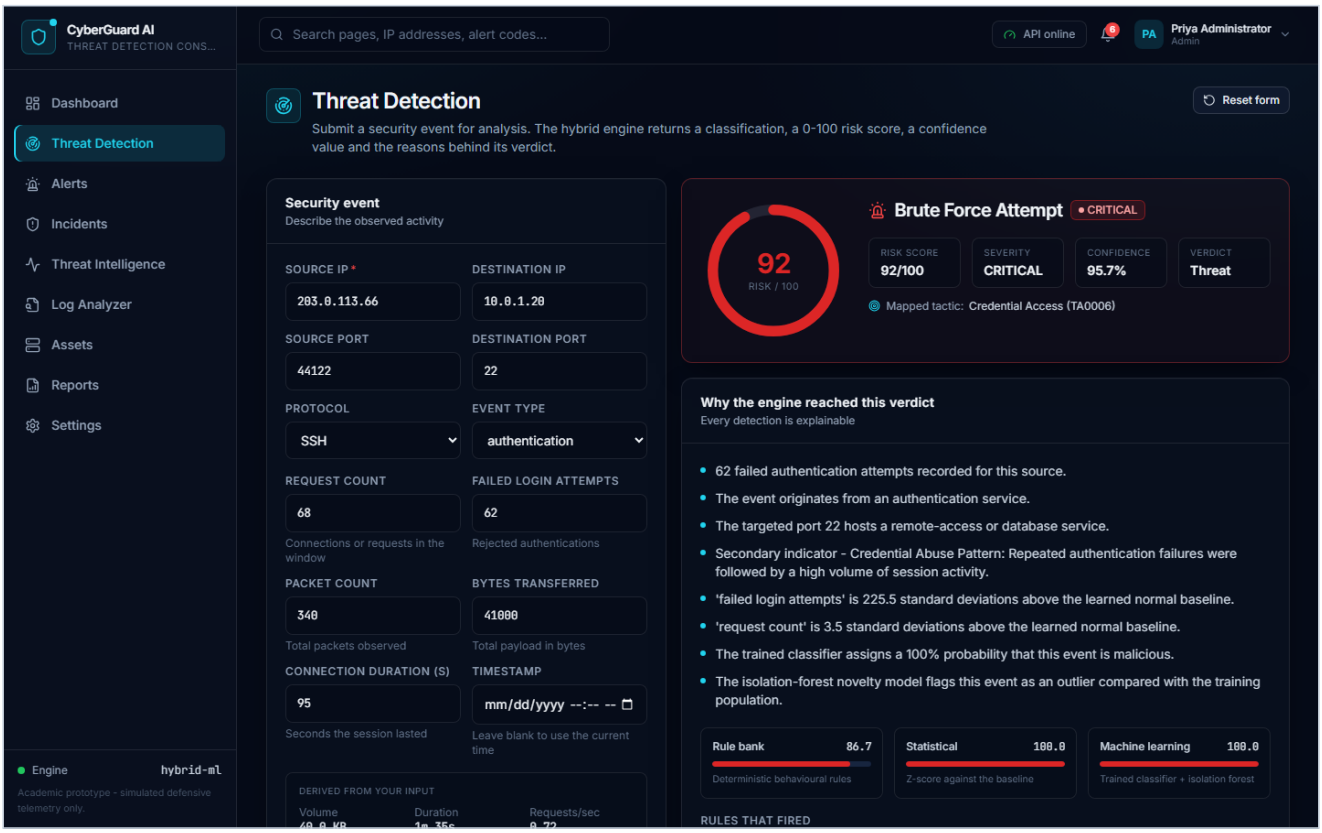
1. Login - demo account cards, with passwords redacted for publication.



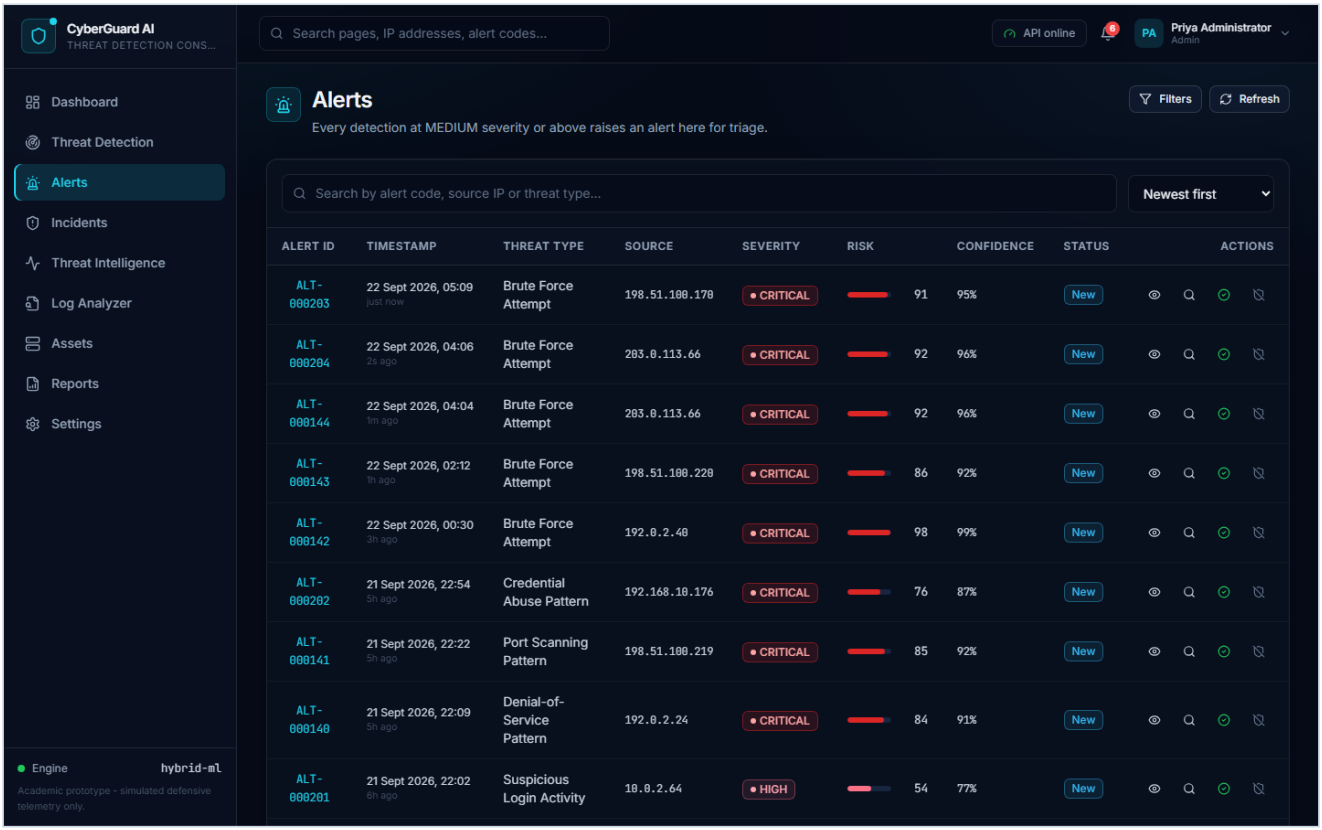
2. Dashboard - 8 KPIs, threat level, and six charts, every figure read from the database.



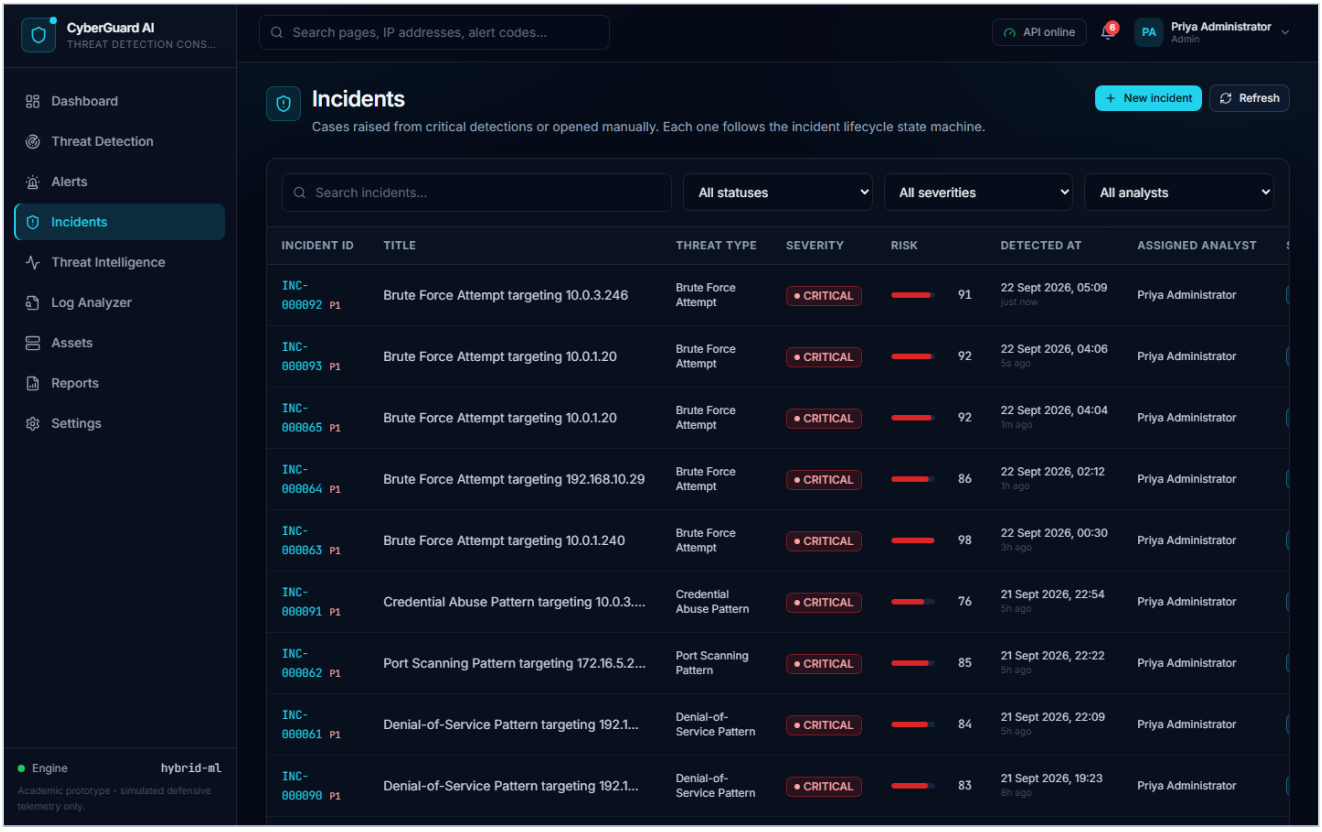
3. Threat Detection - the event form with a sample scenario loaded.



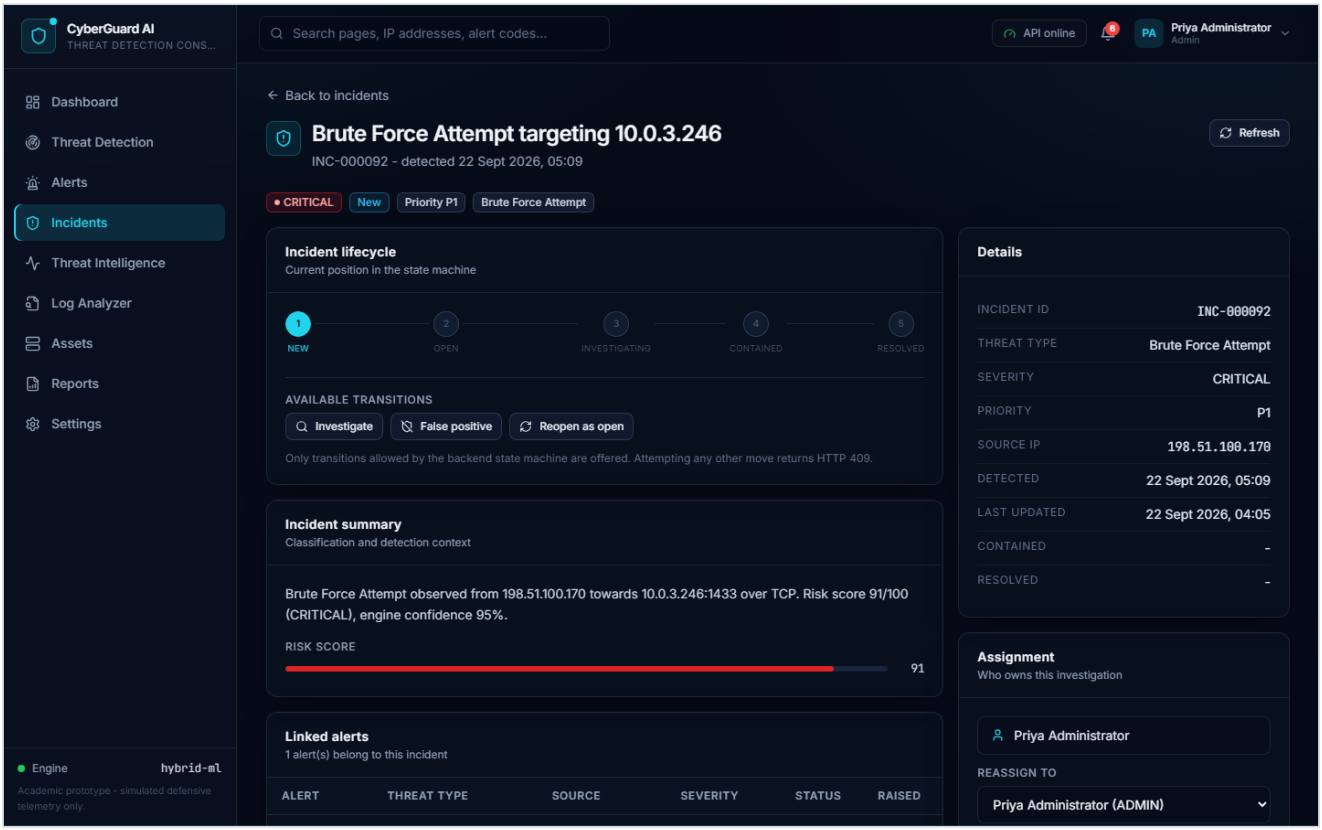
12. Detection result - risk 92/100, CRITICAL, 95.7% confidence, with the reasons and the three detector sub-scores.



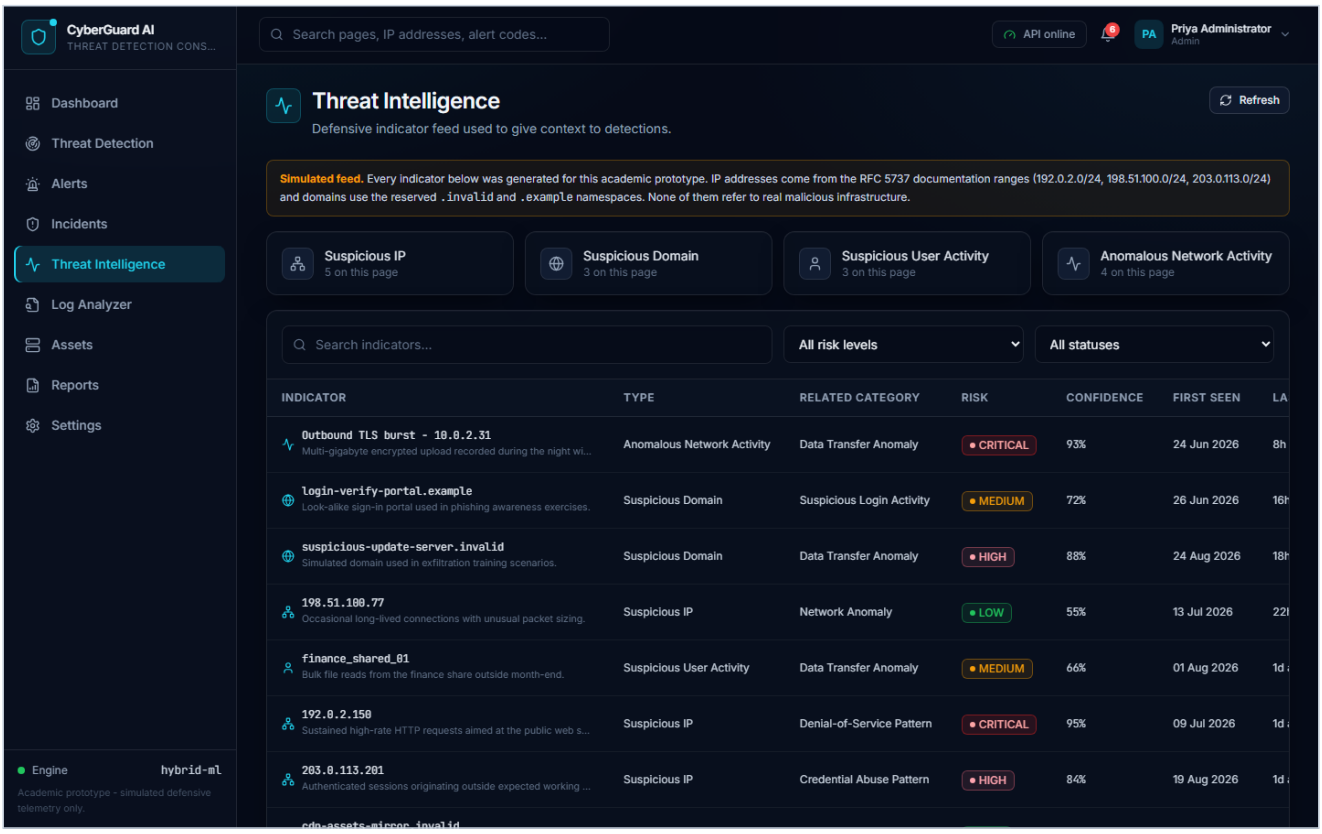
4. Alerts - the triage queue with server-side filtering, search and pagination.



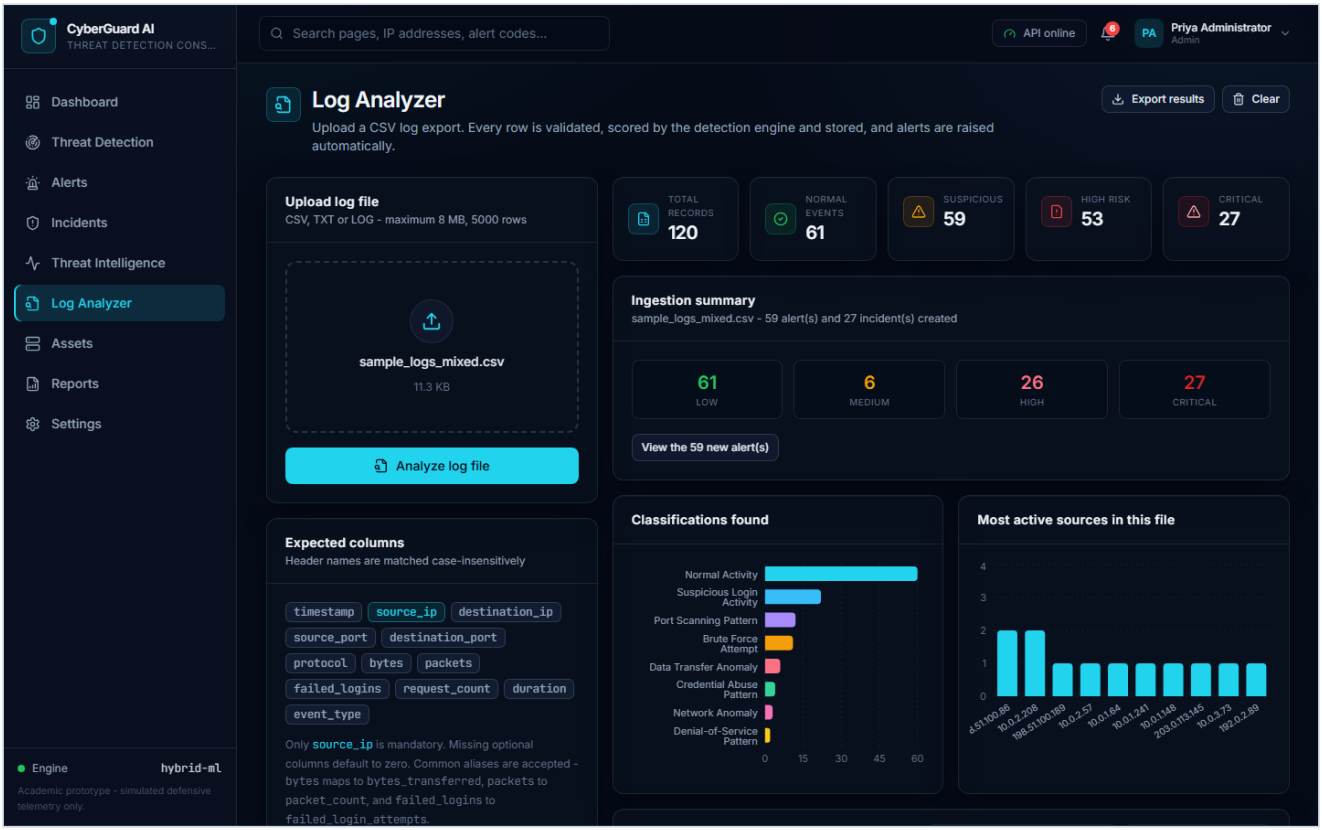
5. Incidents - cases opened automatically at CRITICAL or raised by hand.



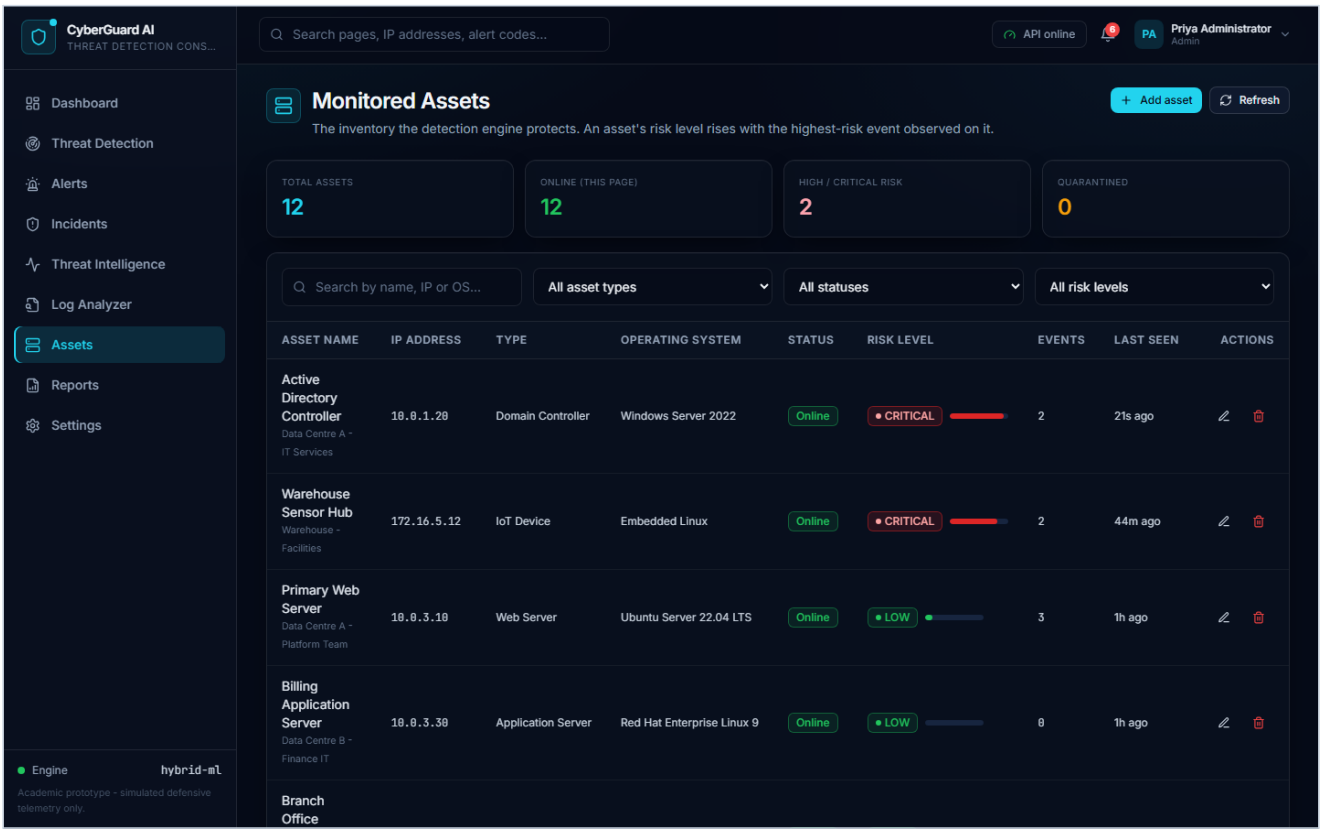
6. Incident details - the lifecycle tracker, and only the transitions the state machine currently permits.



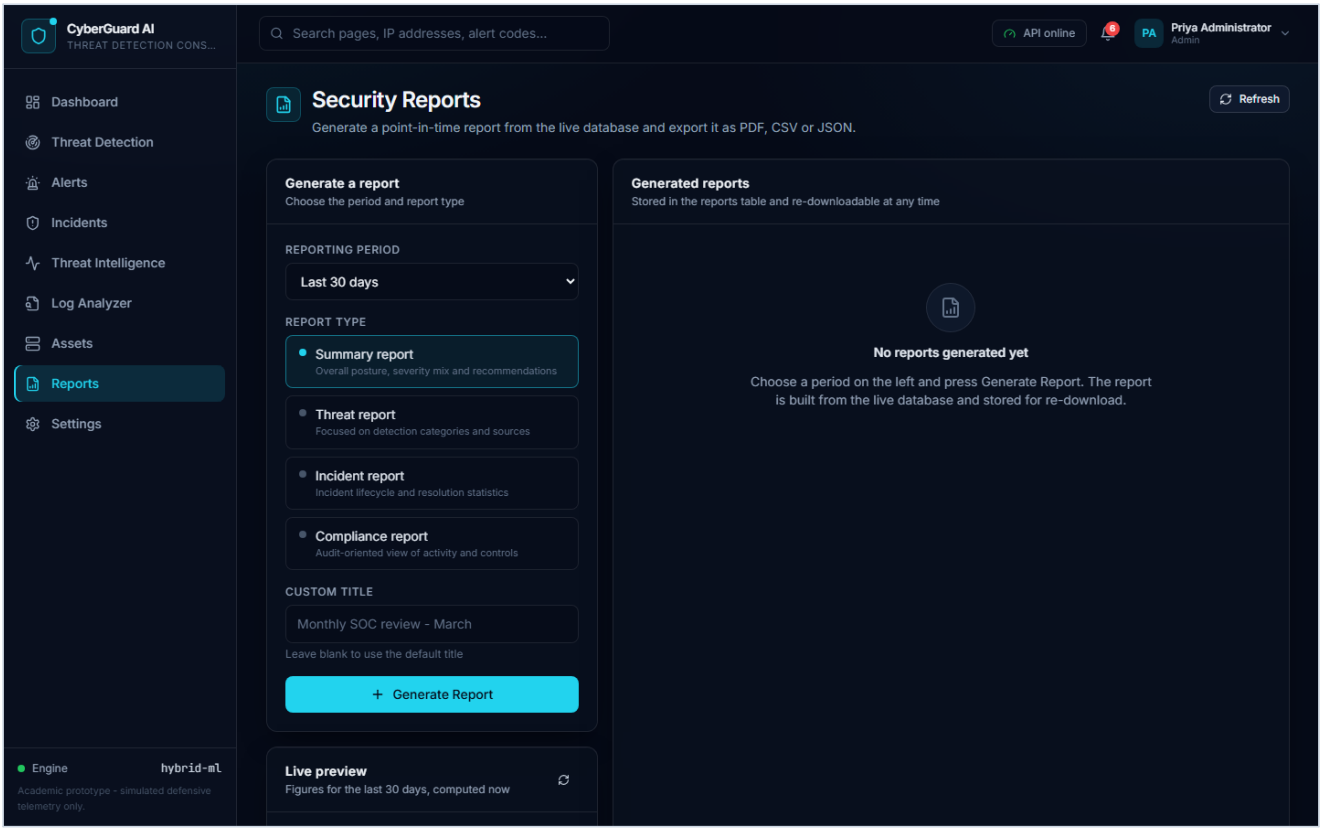
7. Threat Intelligence - a simulated indicator feed, labelled as simulated.



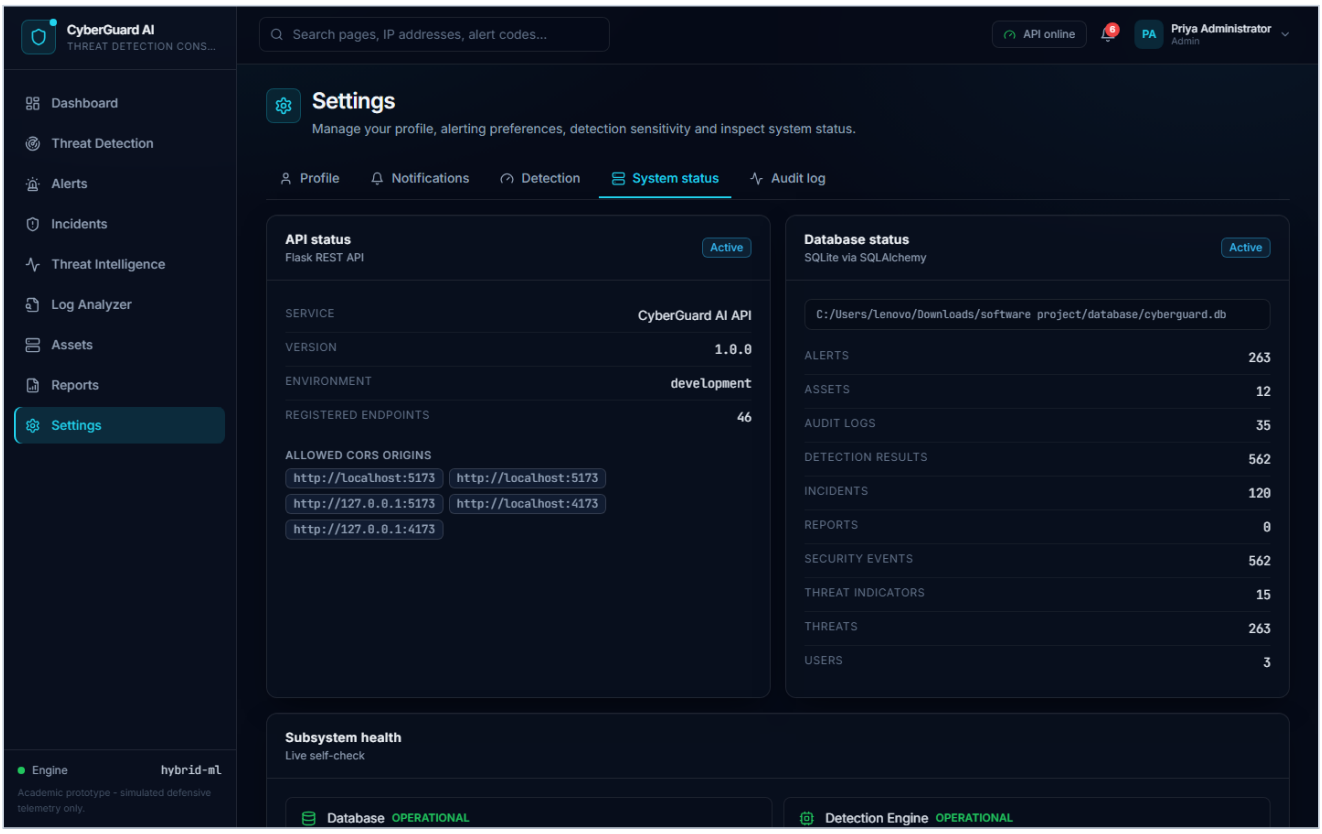
8. Log Analyzer - a 120-row CSV analysed, broken down by severity.



9. Assets - the monitored inventory; risk rises with the worst event seen on each host.



10. Reports - generation form, live preview, and stored reports for re-download.



11. Settings - system status with live per-table row counts.

8 Live demonstration script

A ten-minute walkthrough that touches every syllabus concept against running code. Start both servers before the viva begins and leave them running.

```
Terminal 1:   cd backend
              venv\Scripts\activate
              python run.py

Terminal 2:   cd frontend
              npm run dev

Browser:      http://localhost:5173
```

#	At	Step	What to do and say
1	0:00	**Show that it is real**	Open `http://127.0.0.1:5000/api/health` in a tab. Point at `detection_engine.mode` and `database.connected`. This is a live service, not a mockup.
2	1:00	**Login and dashboard**	Sign in with the Administrator demo card. On the dashboard make two specific points: the Detection Accuracy tile **states its own source** ('Analyst-confirmed precision'), and the Threat Level card **states the basis** for its assessment. The system never shows a bare number.
3	2:00	**Analyse an event - the core demonstration**	Threat Detection → load the sample **'Repeated failed SSH logins'** → Analyze Threat. Talk through the risk score, the severity, the **reasons**, and the **three sub-score bars**. Explain that the rule score is the floor, so a verdict is always anchored to something deterministic. Point out the alert and incident created automatically at the 26 and 76 thresholds. Then submit an **invalid IP** to show validation naming the exact field.
4	5:00	**The state machine - the strongest moment**	Open the incident just created. It is in NEW, and **only three buttons are offered** - Investigate, Reopen as open, False positive. Contain and Resolve are absent. Then attempt the illegal transition directly against the API and show the HTTP 409 naming the allowed set. Finally drive it properly: Investigate → Contain → Resolve, and show the four attributed history entries and the linked alert closing with it.
5	7:00	**CSV upload and error isolation**	Log Analyzer → upload `sample_logs_mixed.csv` and show the severity breakdown. Then upload `sample_logs_invalid.csv`: **1 row analysed, 4 rejected**, each with its line number and reason, and still HTTP 201. Explain that one malformed row must never abort a real log export.
6	8:30	**Reports**	Generate a 30-day summary. Point out that the **recommendations are derived from the actual figures**, not a fixed list. Download the PDF.
7	9:00	**Tests and the honest engineering story**	Run `python -m pytest -q` - 117 passed. Then walk through two of the defects in section 10, especially D1 (severity saturation) and D3 (the performance defect found by measurement).

The one command to have ready

For step 4, paste this into PowerShell to prove the state machine is enforced:

```
$r = Invoke-RestMethod -Uri http://127.0.0.1:5000/api/auth/login -Method Post `
      -ContentType 'application/json' `
      -Body '{"email":"admin@cyberguard.ai","password":"admin123"}'
$T = $r.data.token

# replace 93 with the incident id shown on screen
Invoke-RestMethod -Uri http://127.0.0.1:5000/api/incidents/93 -Method Put `
      -Headers @{ Authorization = "Bearer $T" } `
      -ContentType 'application/json' -Body '{"status":"RESOLVED"}'

# -> HTTP 409 CONFLICT
#    'Invalid lifecycle transition: NEW -> RESOLVED.
#     Allowed from NEW: FALSE_POSITIVE, INVESTIGATING, OPEN.'
```

9 Anticipated viva questions

Q. Why did you choose an Agile/Incremental process model rather than Waterfall?

A. Because the detection logic could not be fully specified in advance - it had to be calibrated against generated data and judged by looking at the output. Waterfall assumes requirements are frozen after phase 1, and that assumption failed in practice: the risk-fusion formula and the severity distribution both had to be redesigned after we saw the first seeded dashboard. A Waterfall process would not have surfaced that defect until after the documentation was written.

Q. Agile de-emphasises documentation, but this lab requires it. How did you reconcile that?

A. By adding documentation to the Definition of Done for every increment. A feature is not complete until the affected document is updated. That is why the state machine in the design document and the transition table in the code cannot drift apart - changing one without the other fails the increment.

Q. How does the detection engine actually decide?

A. Eighteen features are extracted from eleven observed fields. Seven behavioural rules each return a score and reasons; the highest-scoring rule wins the classification. A statistical baseline computes z-scores against learned normals. If trained models are present, a RandomForest and an IsolationForest also score the event. The three are fused with a weighted blend, with the rule score acting as a floor, then scaled by the analyst's sensitivity and mapped to a severity band.

Q. Why is the rule score a floor rather than just another term in the average?

A. So that the statistical and machine-learning detectors cannot single-handedly drive an event to CRITICAL. An unexplained outlier is worth a human glance - MEDIUM - but not an incident. Without the floor the queue fills with unactionable 'critical' noise, which is exactly what happened in the first version.

Q. What happens if scikit-learn is not installed?

A. The engine reports `hybrid-rules` mode and continues using the rule bank plus the default statistical baseline. The model loading is wrapped in a try/except, so even a corrupt artefact degrades rather than crashes. This is proved by a test - `test_engine_works_without_any_trained_model` - not merely claimed. You can demonstrate it by renaming the `ml_model/` folder and restarting.

Q. Your model reports 99.9% accuracy. Is that believable?

A. No, and I would not present it as a detection result. It is measured on a synthetic hold-out split whose classes are well separated by construction, so the score reflects that separability rather than real-world performance. The system has never analysed a real network capture. That caveat is written into the model metadata file and the project report.

Q. How do you know the severity bands are correct?

A. They are asserted at the boundaries by a test - `test_severity_bands_match_the_specification` checks 0, 25, 26, 50, 51, 75, 76 and 100. The constants live in one place in `detector.py` and the documentation describes that constant.

Q. Is the ER diagram actually what the database looks like?

A. Yes, and you can verify it live: `GET /api/system/status` returns the real per-table row counts from the running database. The models in `app/models/` are the schema - `db.create_all()` builds the tables from them.

Q. Why is there duplicated severity data across three tables? That breaks normalisation.

A. It is a deliberate temporal snapshot. An alert records the severity as assessed when it was raised. If an analyst changes their detection sensitivity later, historical detections must keep their original scores, otherwise past charts would silently rewrite themselves. For a security tool, auditability outweighs strict normalisation - and the exception is documented rather than accidental.

Q. Why does ThreatIndicator have no foreign key?

A. Indicators are correlated to events by value, not by referential integrity. A foreign key would mean an indicator could only exist once matching evidence existed, which inverts how an intelligence feed works - the feed is populated before the matching traffic appears.

Q. What stops an analyst putting an incident into an invalid state?

A. A dictionary in `models/incident.py` defines every legal transition. The service validates against it and raises a 409 if the move is not permitted. Separately, the API returns the currently legal set on every read and the page renders a button only for each - so the interface cannot even offer an illegal move. Two guards, one source of truth.

Q. Why do alerts and incidents have different state machines?

A. An alert may legitimately be resolved straight from NEW, because an experienced analyst can recognise a known-benign pattern at a glance. An incident exists precisely because something needs investigation, so allowing it to skip straight to RESOLVED would make its audit trail meaningless.

Q. What happens to the alerts when an incident is resolved?

A. They are closed with it - RESOLVED if the incident was resolved, FALSE_POSITIVE if it was dismissed. That cascade is in `incident_service.transition` and is covered by `test_resolving_an_incident_closes_its_alerts`.

Q. How does a CSV upload handle a malformed row?

A. Each row is validated independently. A failure is recorded as a line number and a reason, and the loop continues. A four-row file with two bad rows returns HTTP 201 with one record analysed and two rejections reported. Rejecting an entire real log export because of one bad line would make the feature unusable.

Q. Is the CSV upload atomic?

A. Yes. Each row is persisted with `commit=False` and a single commit follows the loop, so a file either lands completely or not at all. There is no state where some rows produced alerts and the rest were lost.

Q. How is authentication implemented, and is it secure?

A. Passwords are stored as salted PBKDF2 hashes via Werkzeug. Sessions are signed, expiring tokens created with itsdangerous. It is deliberately simple and I would not call it production-grade: tokens cannot be revoked before expiry, there is no rate limiting, and there is no HTTPS in this setup. Those are listed as known limitations rather than glossed over.

Q. Does the login endpoint leak whether an account exists?

A. No. An unknown email and a wrong password return a byte-identical message and the same 401 status. There is a test asserting the two messages are equal.

Q. How do you prevent SQL injection?

A. Every query is built through SQLAlchemy expression objects, which parameterise values. There is no string-built SQL anywhere in the repository - the single raw statement is `db.text('SELECT 1')`, a constant liveness probe with no interpolation.

Q. How many tests are there and what do they cover?

A. 117 automated cases across five modules, all passing. They use equivalence partitioning, boundary value analysis, state-transition testing and a few property assertions. Every functional requirement has at least two cases, and every requirement has at least one negative, error-path case.

Q. Why can the detection engine be tested without a database?

A. Because `app/ml/` imports nothing from Flask, SQLAlchemy or the rest of the application. That isolation is a deliberate architectural decision, and the 18 engine tests construct a detector directly with no application context.

Q. What is the difference between the dashboard and the analytics endpoint?

A. The dashboard endpoint returns what one screen needs - totals, six chart series, threat level, health and the activity feed. The analytics endpoint returns deeper aggregates: averages, mean time to resolve, hourly distribution, ingestion channels and status breakdowns. Both are read-only; neither writes anything, so refreshing cannot alter the data being reported.

Q. Are the dashboard numbers hard-coded in React?

A. No, and it is verifiable: a test compares the dashboard totals with the corresponding list endpoints and asserts they are equal. Analysing a new event and returning to the dashboard increases the totals.

Q. How would you scale this beyond a prototype?

A. The first real ceiling is SQLite's single-writer model. Because all persistence goes through SQLAlchemy, moving to PostgreSQL is a change to `DATABASE_URL` plus one dialect-specific query. All list endpoints already paginate server-side, and the engine is stateless per call, so it is safe behind multiple workers.

Q. What is the biggest weakness of your detection approach?

A. There is no cross-event correlation - each event is scored in isolation. A slow distributed scan spread thinly across many events would not be detected, because no single event would look unusual. That is the first item on the future-enhancements list.

Q. What did you find hardest?

A. Calibrating the engine. Getting it to classify each attack type correctly was straightforward. Getting it to produce a **useful spread** of severities was not - the first version put every attack in the CRITICAL band and left HIGH completely empty, which made the dashboard meaningless. Fixing it required changing the scoring mathematics, the fusion rule and the data generator together.

Q. Show me something that testing caught that you did not expect.

A. A failing test that was itself wrong. It escalated a CRITICAL alert that already had an auto-created incident, then tried to move it straight to CONTAINED, and the API correctly refused with a 409. The instinct is to change the code; here the code was right and the test encoded a wrong assumption. I corrected the test and added a new one for the property I had misunderstood.

Q. Why is the bundle 824 kB? Is that acceptable?

A. It is a single chunk dominated by Recharts and React, and it gzips to 231 kB. For a locally served prototype that is acceptable, but route-based code splitting is on the enhancement list - it is the obvious next optimisation.

Q. How long does a detection take?

A. 7.1 milliseconds on average in ML mode and 0.033 milliseconds in rules mode, measured over 2000 events. The first measurement was 45 milliseconds; profiling showed the models were loaded with `n_jobs=-1`, so joblib spent longer dispatching a thread pool than predicting on a single row. Forcing `n_jobs=1` at inference gave a 6.4× improvement.

Q. What would you do differently if you started again?

A. Two things. I would build the seeding and the dashboard earlier, because the severity-saturation defect was only visible once real data was rendered - it was invisible in unit tests that checked classification alone. And I would write the engine's regression properties, such as monotonicity of risk with attack intensity, as tests from the very beginning rather than adding them after the defect appeared.

Q. Is this production-ready?

A. No, and the project says so in several places. It is an academic prototype: all telemetry is synthetic, authentication is demonstration-grade, there is no HTTPS or rate limiting, and it runs on a single-process development server. The deployment document lists exactly what would have to change first.

10 Defects found and fixed

> **Why this section matters most**
> Being able to describe a defect you found, diagnosed and fixed is stronger evidence of understanding than any feature list. Each entry below is a real problem encountered during development, with the diagnosis and the fix.

D1 - Severity saturation (the significant one)

Symptom. After the first end-to-end build, the seeded dashboard showed LOW 177, MEDIUM 21, **HIGH 0**, CRITICAL 140. Every simulated attack landed in the top band and the HIGH band was completely empty, so the severity chart and the alert queue conveyed no prioritisation at all.

Diagnosis. Four compounding causes:

- The rule bank used **linear** magnitude terms, so 20 failed logins and 2000 both hit the ceiling of 100.
- The statistical scorer mapped the combined z-score linearly, so any attack-shaped event saturated at 100.
- The fusion floor was `0.85 × max(rule, anomaly, ml)`, which let the statistical detector alone drive an event to CRITICAL.
- The data generator produced every attack at full intensity, which is not how real telemetry is distributed.

Fix.

Change	File
Logarithmic magnitude terms, e.g. 38 + 30·log10(failed/4)	ml/rules.py
Saturating exponential mapping 100 · (1 - e^(-c/9))	ml/anomaly.py
Floor changed to 0.85 × **rule score only**	ml/detector.py
Low/medium/high intensity tiers, weighted towards mild	ml/generate_dataset.py

Result. LOW 57.7%, MEDIUM 9.8%, HIGH 15.5%, CRITICAL 17.0% - a realistic pyramid with all four bands populated.

Prevention. Two regression tests make this impossible to reintroduce silently: one asserts all four bands are populated across 400 generated events, the other asserts the risk score is monotonic in attack intensity.

D2 - Wrong HTTP status for a wrong password

Symptom. A wrong password shorter than six characters returned HTTP 422, a validation error, instead of 401. **Diagnosis.** The registration password-strength rule was being applied at sign-in. Besides the wrong status code it leaked information: a 422 told an attacker the password they tried was too short to be the real one. **Fix.** Login now checks only that a password was supplied; the strength rule applies to password changes.

D3 - Machine-learning inference 6.4× slower than necessary

Symptom. Performance measurement showed ML mode averaging 45.353 ms per event against 0.025 ms for rules mode. **Diagnosis.** Timing the sub-calls isolated it to `RandomForest.predict_proba` at 43 ms for a **single** sample. The models were saved with `n_jobs=-1`, correct for fitting, but at inference the engine scores one event at a time and joblib spent far longer dispatching to a thread pool than doing the work. **Fix.** Force `n_jobs = 1` on the loaded models; training still uses all cores. **Result.** 45.353 ms → 7.083 ms, and the test suite dropped from 78 s to 51 s as a side effect.

D4 - A failing test that was itself wrong

Symptom. A test asserting that resolving an incident closes its alerts failed. **Diagnosis.** The test escalated a CRITICAL alert that **already had an auto-created incident in NEW**, then tried to move it straight to CONTAINED. The API correctly refused with 409, because NEW → CONTAINED is not a legal transition. **Resolution.** The code was right and the test encoded a wrong assumption. The test was corrected to investigate first, and a new test was added for the property that had been misunderstood - that escalation is idempotent and returns the existing incident rather than creating a duplicate.

D5 - Build failure on an invalid Tailwind opacity

`border-white/12` is not on Tailwind's default opacity scale, so the production build failed. Replaced with `/15` throughout. Minor, but a good illustration of why a build step belongs in the Definition of Done.

D6 - Every timestamp displayed 5.5 hours in the past

Symptom. The dashboard showed 'Updated 5h ago' on a payload generated one second earlier, and every alert and incident timestamp was similarly shifted. **Diagnosis.** The backend stores and emits naive UTC (`2026-09-21T22:35:51`, with no zone designator). A date-time string without a designator is parsed by the browser as **local** time, so on a UTC+5:30 machine every value was shifted by the local offset. **Fix.** A single `parseTimestamp` helper in `utils/format.js` marks zone-less date-time strings as UTC before parsing; the four formatting functions all route through it. Date-only strings, used by the chart series, are already treated as UTC by the specification and are left untouched.

11 Limitations and future work

Volunteer these before you are asked

An examiner will find at least one of these. Naming them first demonstrates judgement; being caught by one looks like an oversight.

Known limitations

Area	Limitation
Security	Tokens cannot be revoked before expiry - there is no server-side session store. No rate limiting on login. No HTTPS. Role checks are coarse. No penetration testing or dependency scanning was performed.
Data	All telemetry is synthetic; the system has never analysed a real network capture. The reported model accuracy is measured on synthetic data whose classes are separable by construction.
Detection	No cross-event correlation - each event is scored in isolation, so a slow distributed scan would not be detected. Indicators are not automatically matched against events.
Scale	SQLite allows one writer at a time. CSV uploads are capped at 5000 rows because parsed rows are held in memory. Single-process development server, no clustering.
Testing	No automated frontend tests - React is verified manually and by the build. No cross-browser matrix, no real-device testing, no automated accessibility audit.

Future enhancements, in priority order

Priority	Enhancement	Why it is next
High	Cross-event correlation by source over a time window	The most significant detection gap
High	Automatic indicator matching against incoming events	Makes the intelligence feed operational, not just informational
High	Frontend test suite (Vitest + React Testing Library)	Closes the largest gap in the test strategy
Medium	Server-side session store for token revocation	Removes a stated security limitation
Medium	Rate limiting on authentication and upload	Removes a stated security limitation
Medium	Route-based code splitting	The bundle is currently one chunk
Medium	PostgreSQL deployment guide	Lifts the single-writer ceiling
Medium	Real PCAP or syslog ingestion	Would allow genuine calibration
Low	Light theme	Preference is stored but only dark ships
Low	WebSocket live feed	The dashboard currently polls every 45 seconds

12 Quick reference card

Startup

```
cd backend                          cd frontend
venv\Scripts\activate               npm run dev
python run.py                       -> http://localhost:5173

python seed_database.py             rebuild the demo dataset
python -m pytest -q                 run the 117 tests
python -m app.ml.train_model        retrain the optional models
python capture_screenshots.py       re-capture the 12 screens
```

Numbers to memorise

Constant	Value
Severity LOW	0 - 25
Severity MEDIUM	26 - 50
Severity HIGH	51 - 75
Severity CRITICAL	76 - 100
Alert threshold	risk ≥ 26
Incident threshold	risk ≥ 76
Sensitivity range	0.5 - 1.5
Confidence range	0.50 - 0.99
Upload ceiling	8 MB / 5000 rows

Metric	Value
Features	18
Rules	7
Categories	8
Entities	10
Endpoints	46
Screens	12
Tests	117 passing
Detection (ML)	7.083 ms mean
Detection (rules)	0.033 ms mean

The eight detection categories

Normal Activity • Brute Force Attempt • Port Scanning Pattern • Suspicious Login Activity • Network Anomaly • Data Transfer Anomaly • Denial-of-Service Pattern • Credential Abuse Pattern

Demonstration credentials

Role	Email	Password
Administrator	admin@cyberguard.ai	admin123
Security Analyst	analyst@cyberguard.ai	analyst123
Viewer	viewer@cyberguard.ai	viewer123

These are seeded credentials for a local academic prototype, stored as salted PBKDF2 hashes and published in the project README. They are included here because they are needed to sign in during the demonstration. They are not real secrets and grant no access to anything beyond a local SQLite file.

Where everything lives

Looking for	Path
Detection engine	backend/app/ml/
State machine	backend/app/models/incident.py
Severity bands	backend/app/ml/detector.py
Validation rules	backend/app/utils/validators.py
Business logic	backend/app/services/
API routes	backend/app/routes/
Tests	backend/tests/
Screens	frontend/src/pages/
The only network module	frontend/src/services/api.js
SE documents	documentation/ (23 files)
Screenshots	documentation/screenshots/ (12 files)
Sample CSV files	sample_data/ (4 files)

The closing line

"CyberGuard AI is a working full-stack security platform where the documentation and the implementation are the same system viewed from two angles. The state transition diagram is the transition table the API enforces; the ER diagram is the schema SQLAlchemy creates; the severity bands in the data flow diagram are the constants the engine applies. It is an academic prototype on simulated telemetry, and every figure it reports states where it came from."